

REVIEW OF PREVIOUS LECTURE

Measures for evaluating classifiers:

$$accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{P} = \frac{TP}{TP + FN}$$

$$F = 2 \times \frac{precision \times recall}{precision + recall}$$

To evaluate a 'score' for in-classness:

Compute AUROC: ROC curve plots TPR , FPR for each threshold.

MEASURES FOR EVALUATING CLASSIFIERS

Multiclass systems: Accuracy easy! How to generalize precision/recall?

Ex: Confusion matrix for OCR of single digits

		Predicted									
Actual	Digits	0	1	2	3	4	5	6	7	8	9
	0	351	0	5	4	2	7	2	1	6	0
	1	0	254	0	0	2	0	0	1	1	2
	2	1	1	166	4	5	1	3	2	2	1
	3	1	2	4	142	0	5	0	1	4	0
	4	3	3	8	1	180	3	2	5	4	4
	5	0	0	3	11	0	140	3	0	7	1
	6	0	2	2	0	4	0	158	0	1	0
	7	0	0	2	2	1	0	0	132	2	1
	8	2	1	8	0	0	0	2	1	137	1
	9	1	1	0	2	6	4	0	4	2	167

MEASURES FOR EVALUATING CLASSIFIERS

Multiclass systems: Accuracy easy! How to generalize precision/recall?

Ex: Confusion matrix for OCR of single digits

Predicted

Digits	0	1	2	3	4	5	6	7	8	9
0	351	0	5	4	2	7	2	1	6	0
1	0	254	0	0	2	0	0	1	1	2
2	1	1	166	4	5	1	3	2	2	1
3	1	2	4	142	0	5	0	1	4	0
4	3	3	8	1	180	3	2	5	4	4
5	0	0	3	11	0	140	3	0	7	1
6	0	2	2	0	4	0	158	0	1	0
7	0	0	2	2	1	0	0	132	2	1
8	2	1	8	0	0	0	2	1	137	1
9	1	1	0	2	6	4	0	4	2	167

Actual

$$precision_i = C[i, i] / \sum_{j=1}^d C[j, i]$$

E.g. $precision_0 = 351/359$

$$recall_i = C[i, i] / \sum_{j=1}^d C[i, j]$$

E.g. $recall_0 = 351/378$

EVALUATING VALUE PREDICTION MODELS

Features				Output
x_{11}	x_{12}	$\dots$	x_{1p}	3.2
x_{21}	x_{22}	$\dots$	x_{2p}	4.0
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
x_{n1}	x_{n2}	$\dots$	x_{np}	3.5

Prediction:
Output is a numerical variable

We use x to predict y . Let the predictions be $\hat{y} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$.

EVALUATING VALUE PREDICTION MODELS

Actual values $y = \{y_1, y_2, \dots, y_n\}$. Predictions $\hat{y} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$.

Absolute Error: $\Delta = \hat{y}_i - y_i$

The sign distinguishes $\hat{y} > y$ from $\hat{y} < y$.

Can't aggregate directly (offsetting errors with different signs).

EVALUATING VALUE PREDICTION MODELS

Actual values $y = \{y_1, y_2, \dots, y_n\}$. Predictions $\hat{y} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$.

Absolute Error: $\Delta = \hat{y}_i - y_i$

The sign distinguishes $\hat{y} > y$ from $\hat{y} < y$.

Can't aggregate directly (offsetting errors with different signs).

Relative Error: $(\hat{y}_i - y_i)/y_i$

Unit-less quantity as compared to absolute error.

EVALUATING VALUE PREDICTION MODELS

Actual values $y = \{y_1, y_2, \dots, y_n\}$. Predictions $\hat{y} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$.

Absolute Error: $\Delta = \hat{y}_i - y_i$

The sign distinguishes $\hat{y} > y$ from $\hat{y} < y$.

Can't aggregate directly (offsetting errors with different signs).

Relative Error: $(\hat{y}_i - y_i)/y_i$

Unit-less quantity as compared to absolute error.

Squared Error: $\Delta^2 = (\hat{y}_i - y_i)^2$

Values can be meaningfully summed.

Large errors contribute more $\Rightarrow$ outliers may dominate error.

EVALUATING VALUE PREDICTION MODELS

Good idea to plot the distribution of absolute error values.

What is expected from a good predictor?

- Symmetric distribution centered around zero.

- Bell-shaped (small errors more common than large ones)

- Extreme outliers rare.

EVALUATING VALUE PREDICTION MODELS

Good idea to plot the distribution of absolute error values.

What is expected from a good predictor?

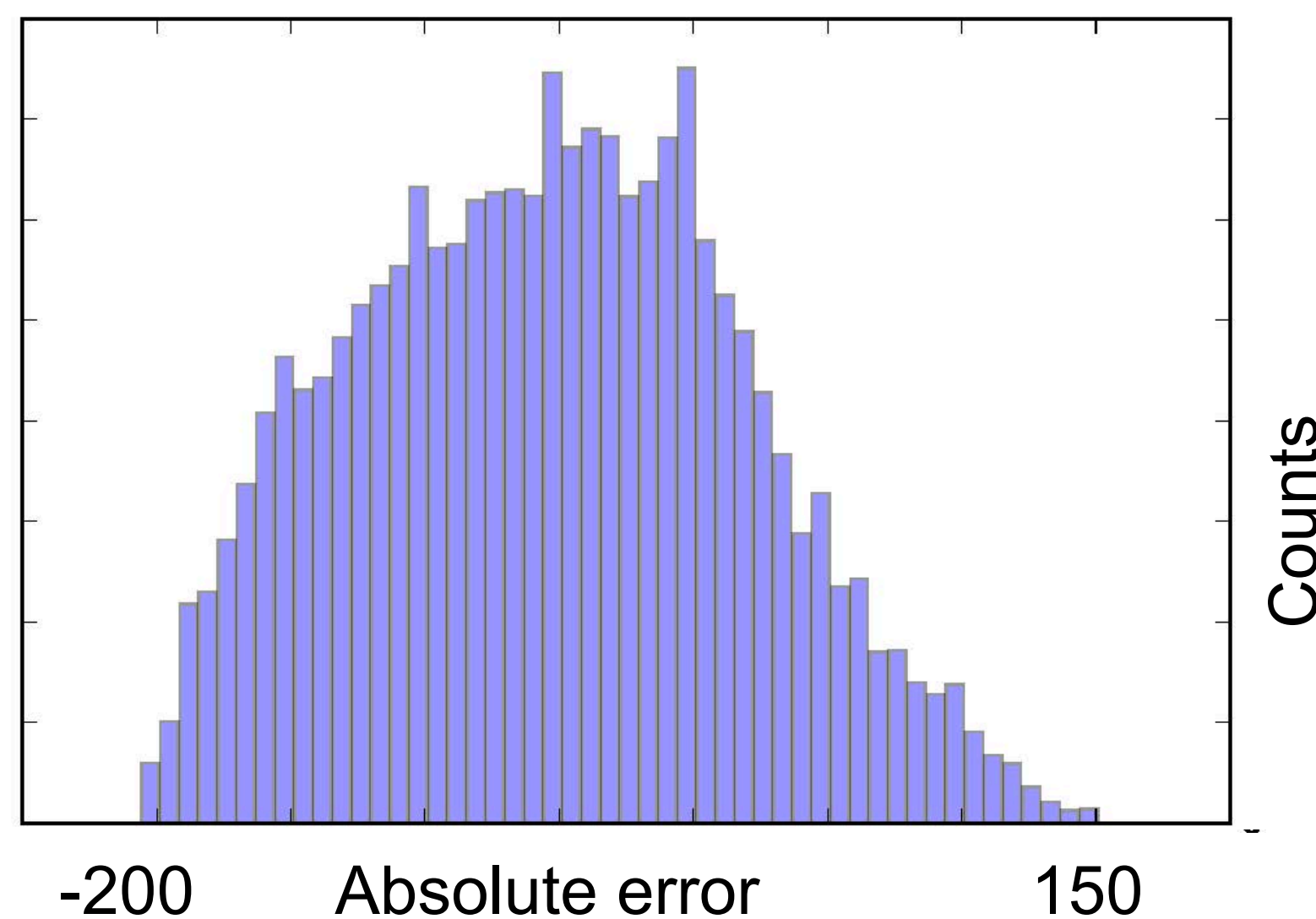
Symmetric distribution centered around zero.

Bell-shaped (small errors more common than large ones)

Extreme outliers rare.

Ex: Document date (1800-2005) predictor from word usage.

Random
model



Why is the left tail fat?

EVALUATING VALUE PREDICTION MODELS

Good idea to plot the distribution of absolute error values.

What is expected from a good predictor?

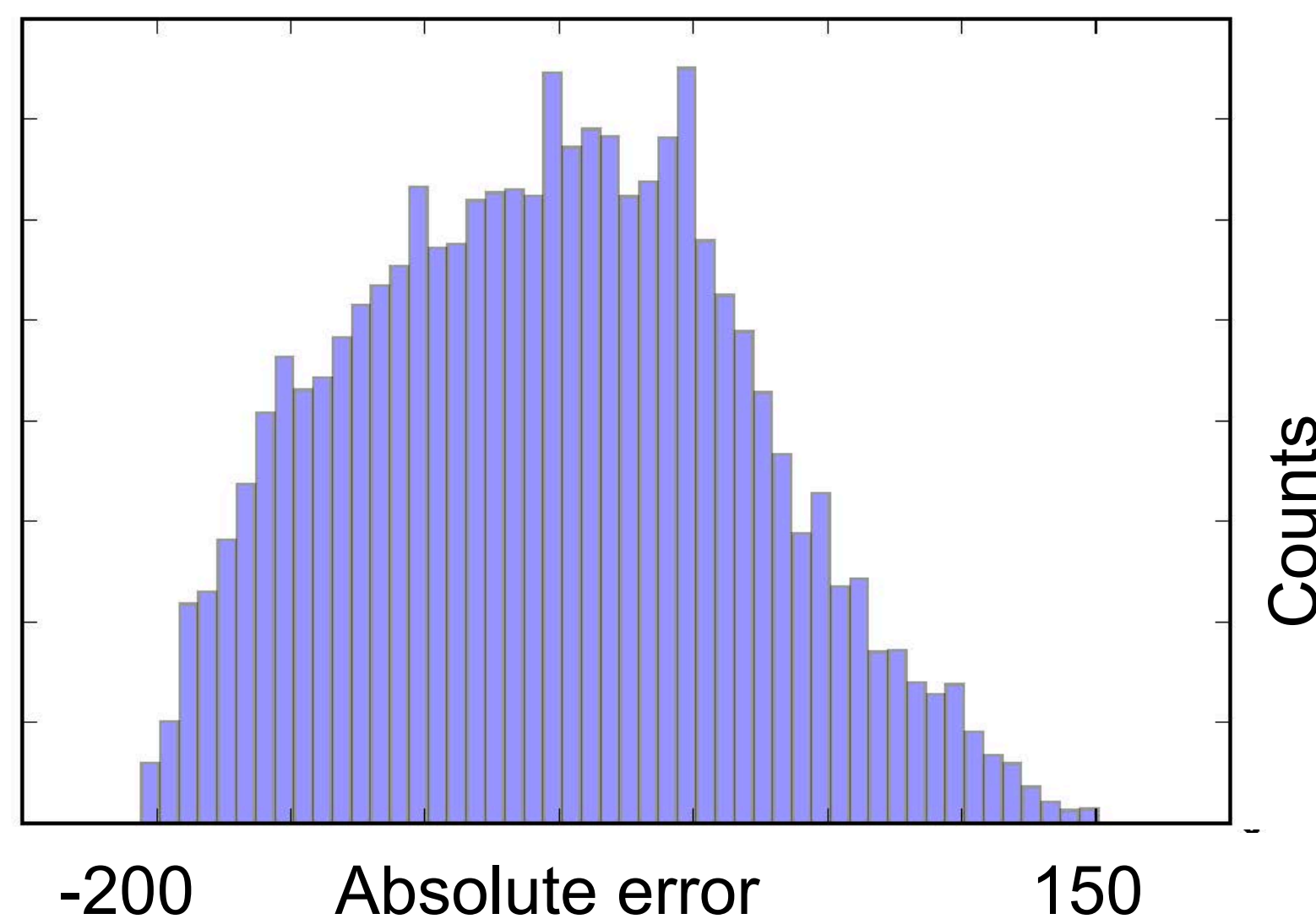
Symmetric distribution centered around zero.

Bell-shaped (small errors more common than large ones)

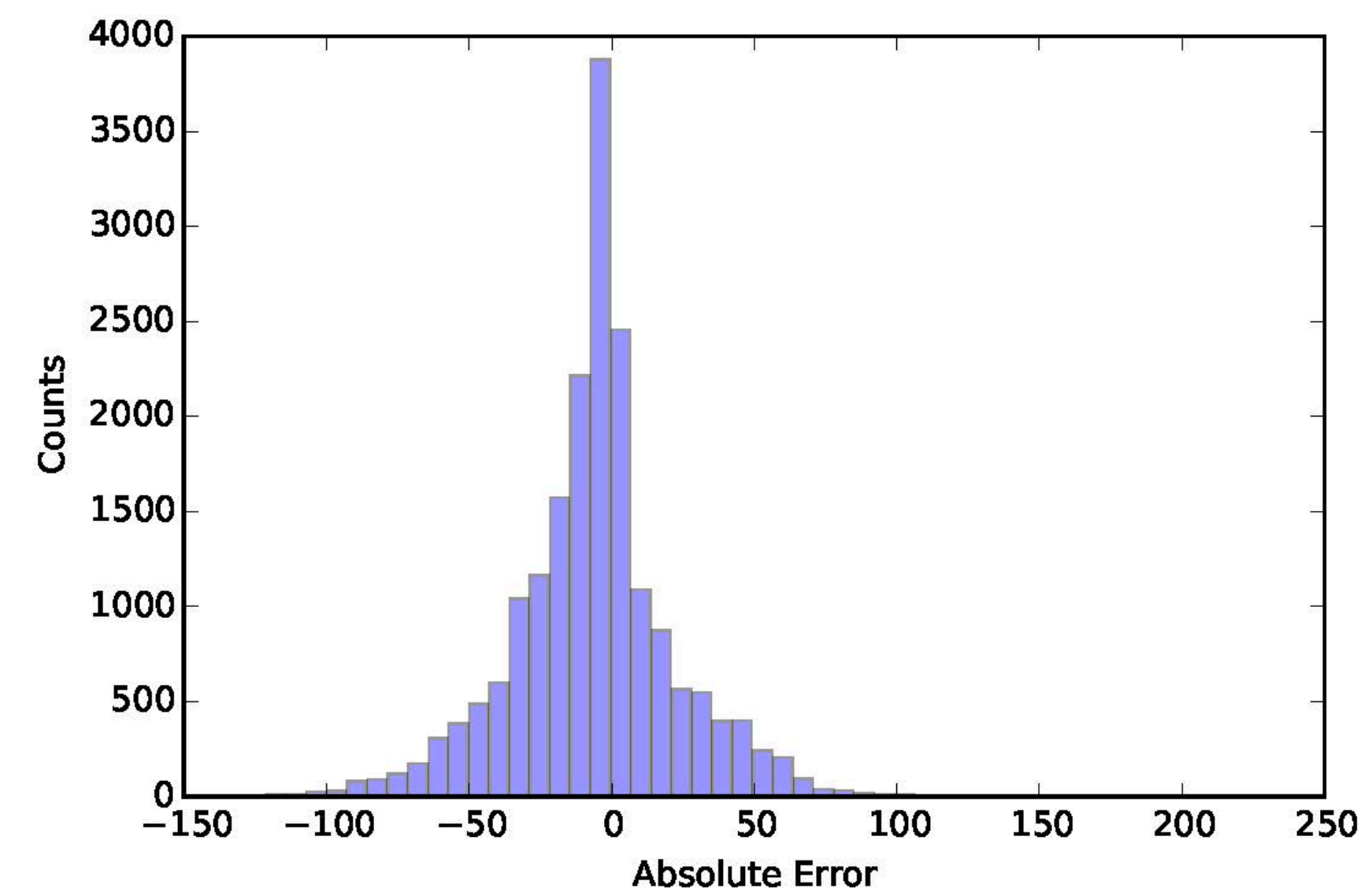
Extreme outliers rare.

Ex: Document date (1800-2005) predictor from word usage.

Random
model



Naive
Bayes
model



EVALUATING VALUE PREDICTION MODELS

We need aggregate summary statistics as well.

Mean Squared Error: $MSE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$

Outliers have disproportionate effect.

Median squared error better for noisy instances.

Root Mean Squared Error: $RMSE(y, \hat{y}) = \sqrt{MSE(y, \hat{y})}$

Magnitude same scale as original values.

Outliers may still be problematic.

PARTITIONING THE DATA

The best way to assess models involve out-of-sample predictions.

PARTITIONING THE DATA

The best way to assess models involve out-of-sample predictions.

Partition the data:



80 % training set

Study the domain and determine parameters of the model.

20 % test set

Never leaks into the model until the end.

TRAINING ERROR VS VALIDATION ERROR

Evaluating on
training data in
terms of accuracy

If performance is
not good $\Rightarrow$
underfitting

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

clf = RandomForestClassifier(n_estimators=2, random_state=0)
# X is the training data
clf.fit(X, y)

# Training error (train and validate on the same data)
y_pred = clf.predict(X)
acc = accuracy_score(y, y_pred)

print(f'Accuracy: {acc:.2f}')
```

Accuracy: 0.97

TRAINING ERROR VS VALIDATION ERROR

```
from sklearn.model_selection import train_test_split

# split into training/validation sets: 70-30
# random_state set to an int for reproducibility
X_train, X_validate, y_train, y_validate = \
    train_test_split(X, y, train_size=0.7, random_state=0)

clf = RandomForestClassifier(n_estimators=2, random_state=0)
clf.fit(X_train, y_train)

# validate with unseen data
y_pred = clf.predict(X_validate)
acc = accuracy_score(y_validate, y_pred)

print(f'Accuracy: {acc:.2f}')
```

Accuracy: 0.91

Split into training and test sets.
Evaluate on validation data in terms of accuracy

If performance is good on training data but not on test data $\Rightarrow$ overfitting

K-FOLD CROSS VALIDATION

By random chance, may have selected a validation set that was not representative of other unseen data that the model might encounter.

```
Validation error from training/validation split #1: 0.82
```

```
Validation error from training/validation split #2: 0.97
```

```
...
```


K-FOLD CROSS VALIDATION

By random chance, may have selected a validation set that was not representative of other unseen data that the model might encounter.

```
Validation error from training/validation split #1: 0.82
```

```
Validation error from training/validation split #2: 0.97
```

```
...
```

Ideally: Assess model performance on several different validation sets before touching the test set.

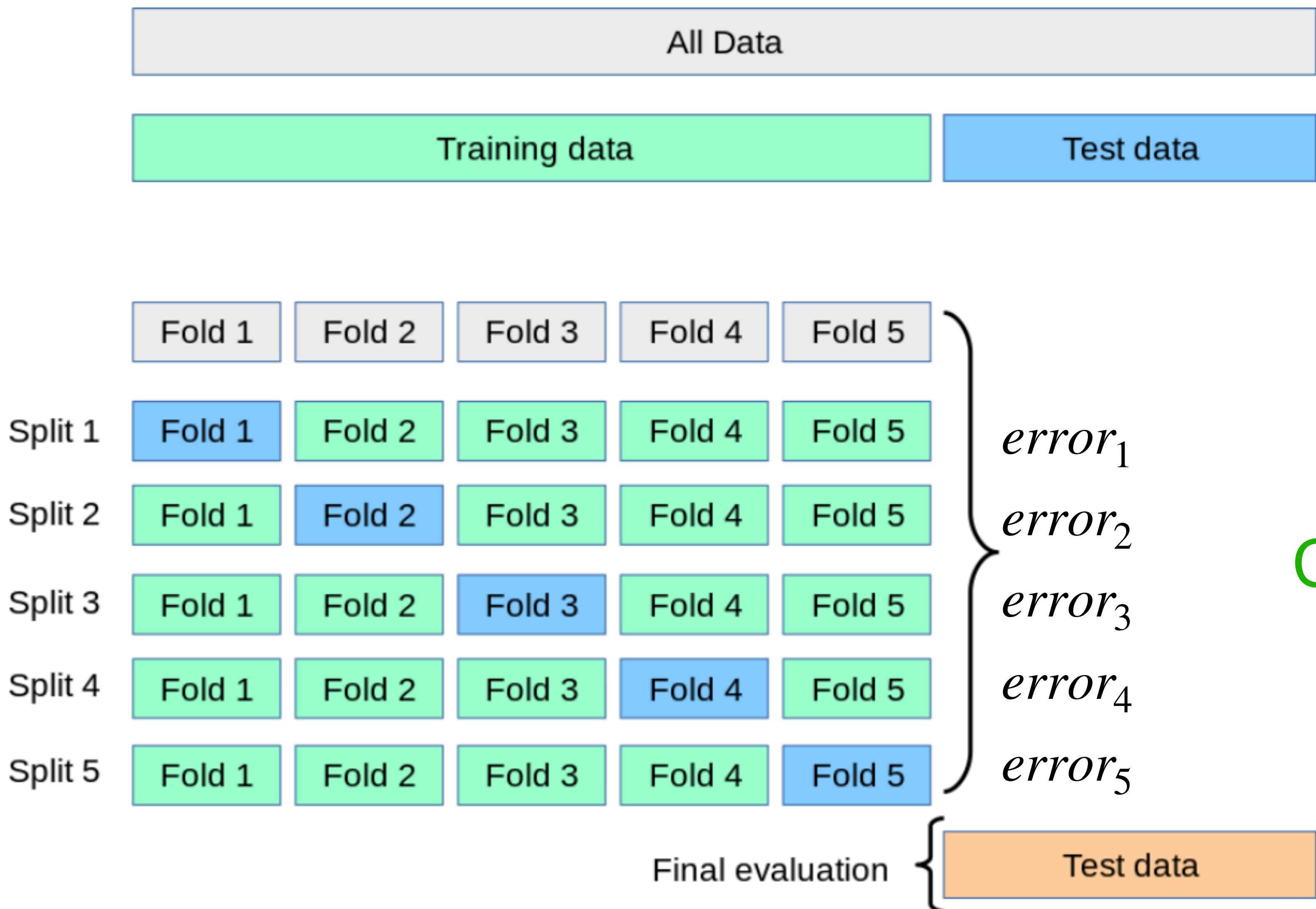
Especially useful for:

Small datasets.

Hyperparameter tuning.

K-FOLD CROSS VALIDATION

Example: 5-fold cross validation



$$CV\ error = 1/5 \sum_{i=1}^5 error_i$$

K-FOLD CROSS VALIDATION: EXAMPLE

```
from sklearn.model_selection import cross_validate  
cv = cross_validate(clf, X, y, scoring='accuracy', cv=3)  
cv["test_score"]
```

```
array([0.98, 0.92, 0.96])
```

```
cv["test_score"].mean()
```

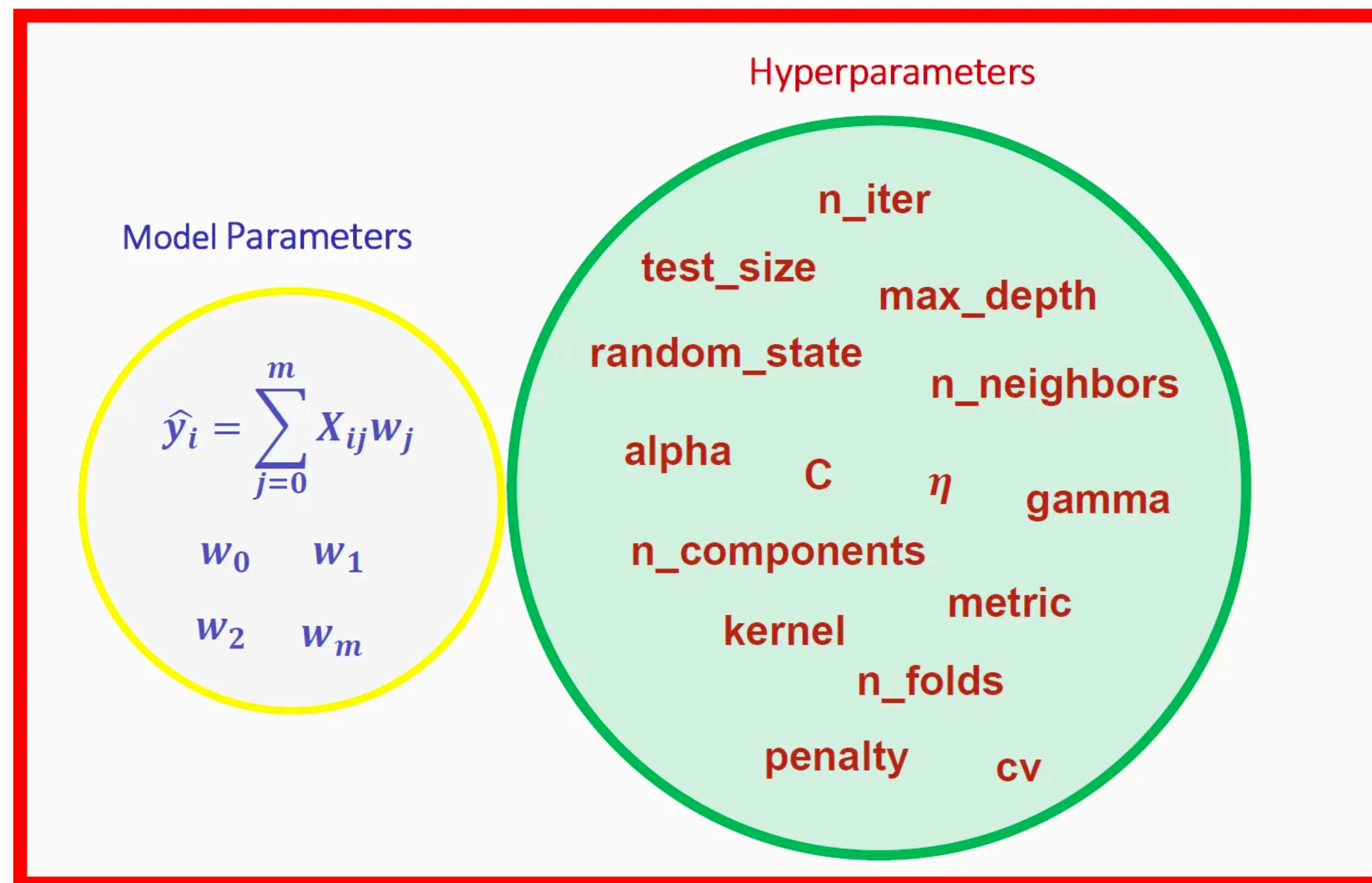
```
0.9533333333333333
```

3-fold cross
validation, find
error (accuracy).

PARAMETERS VS HYPERPARAMETERS

Parameters: Learned during training. Specific to the model being used. **E.g.**, in linear regression, coefficients assigned to each feature.

Hyperparameters: Set by the user before training. Not trained by ML algorithms to find the optimum settings. We must choose it ourselves.



HYPERPARAMETER TUNING

For each hyperparameter setting $h_i \in \{h_0, h_1, \dots, h_r\}$

run cross-validation to compute the CV error for h_i

Select h_i with lowest CV error.

HYPERPARAMETER TUNING

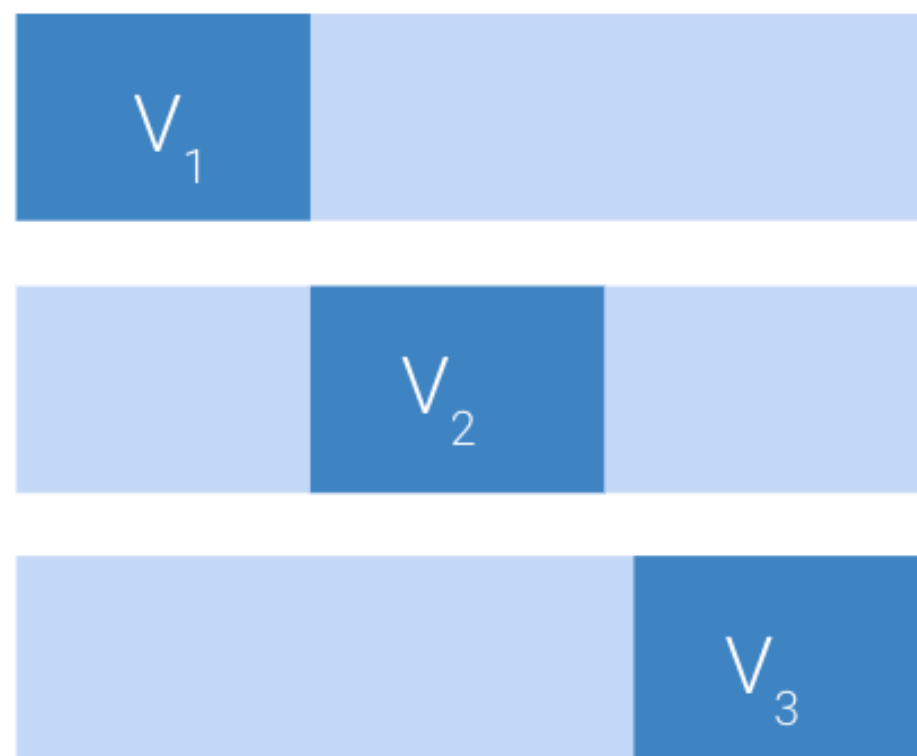
For each hyperparameter setting $h_i \in \{h_0, h_1, \dots, h_r\}$

run cross-validation to compute the CV error for h_i

Select h_i with lowest CV error.

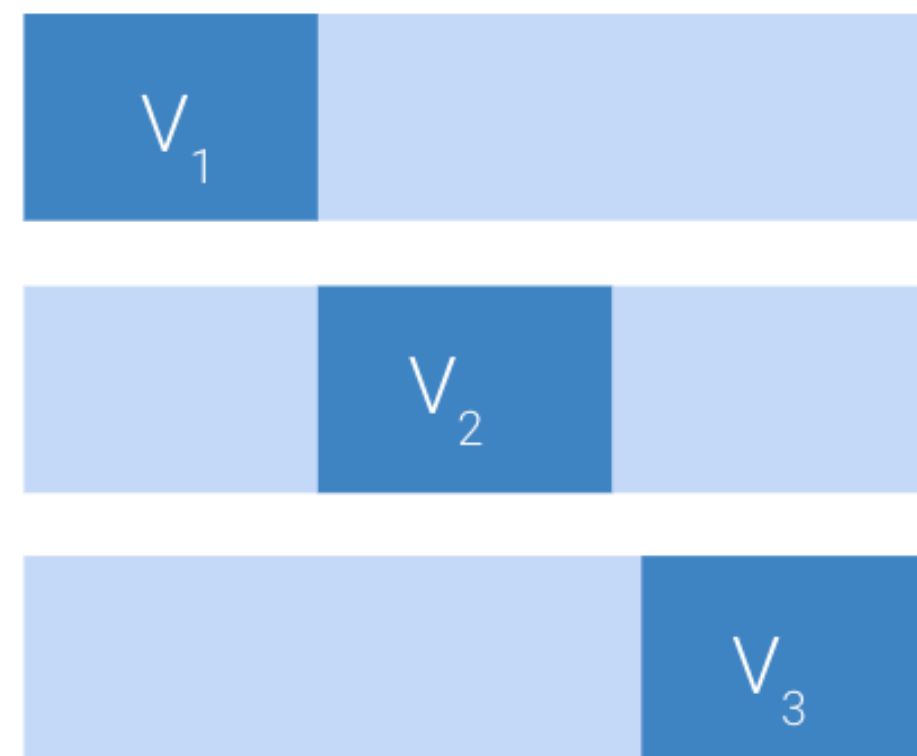
Example: Hyperparameter tuning for $n_estimators$ at settings of 10, 20, 50 with a 3-fold cross validation.

$n_estimators = 10$



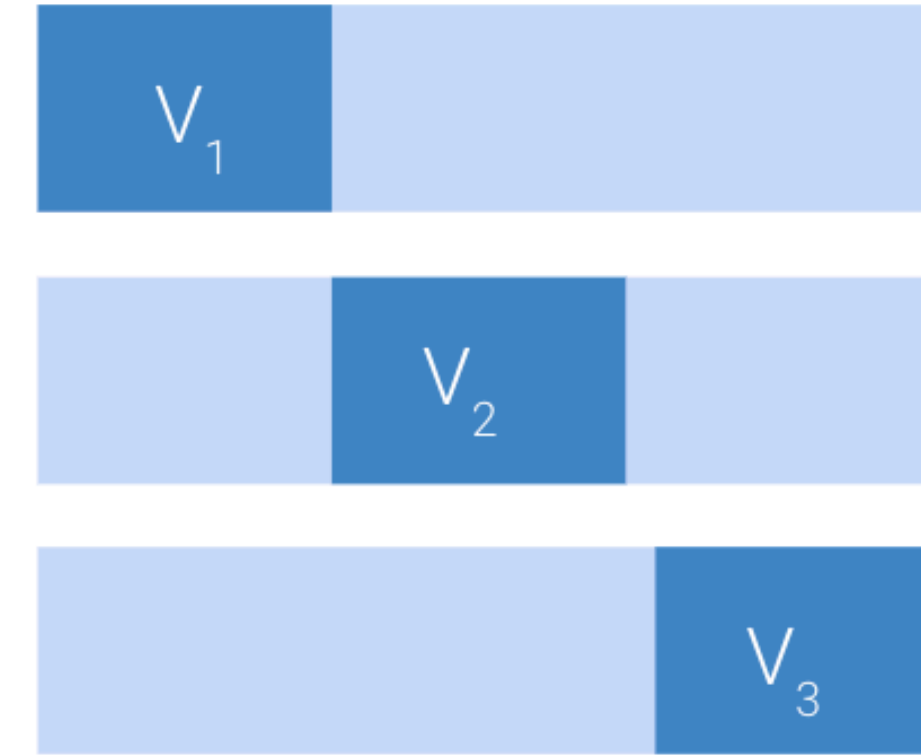
$CV\ error = 6.56$

$= 20$



$CV\ error = 4.22$

$= 50$



$CV\ error = 3.15$

LINEAR ALGEBRA REVIEW

Why?

- Typical data in Data Science:
All the relevant information in one or more data matrices
The rows $\Rightarrow$ items (examples), the columns $\Rightarrow$ features (attributes).
- Many ML algorithms are best understood through linear algebra
Ex: Linear regression reduces to a single formula.

Most of you must have taken a linear algebra course already.
This will just be a review.

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Data:

Features (dimensions)

Items (instances)

x_{11}	x_{12}	$\dots$	x_{1p}
x_{21}	x_{22}	$\dots$	x_{2p}
$\vdots$	$\vdots$	$\begin{smallmatrix} \cdot & & \cdot \\ & \cdot & \end{smallmatrix}$	$\vdots$
x_{n1}	x_{n2}	$\dots$	x_{np}

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- System of equations:

$$\begin{array}{rcl} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n & = & b_1 , \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n & = & b_2 , \\ & \vdots & \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n & = & b_n . \end{array} \quad \longrightarrow \quad \begin{array}{c} A \\ \left(\begin{array}{cccc} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{array} \right) \end{array} \begin{array}{c} x \\ \left(\begin{array}{c} x_1 \\ x_2 \\ \vdots \\ x_n \end{array} \right) \end{array} = \begin{array}{c} b \\ \left(\begin{array}{c} b_1 \\ b_2 \\ \vdots \\ b_n \end{array} \right) \end{array}$$

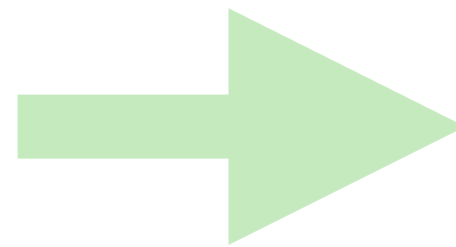
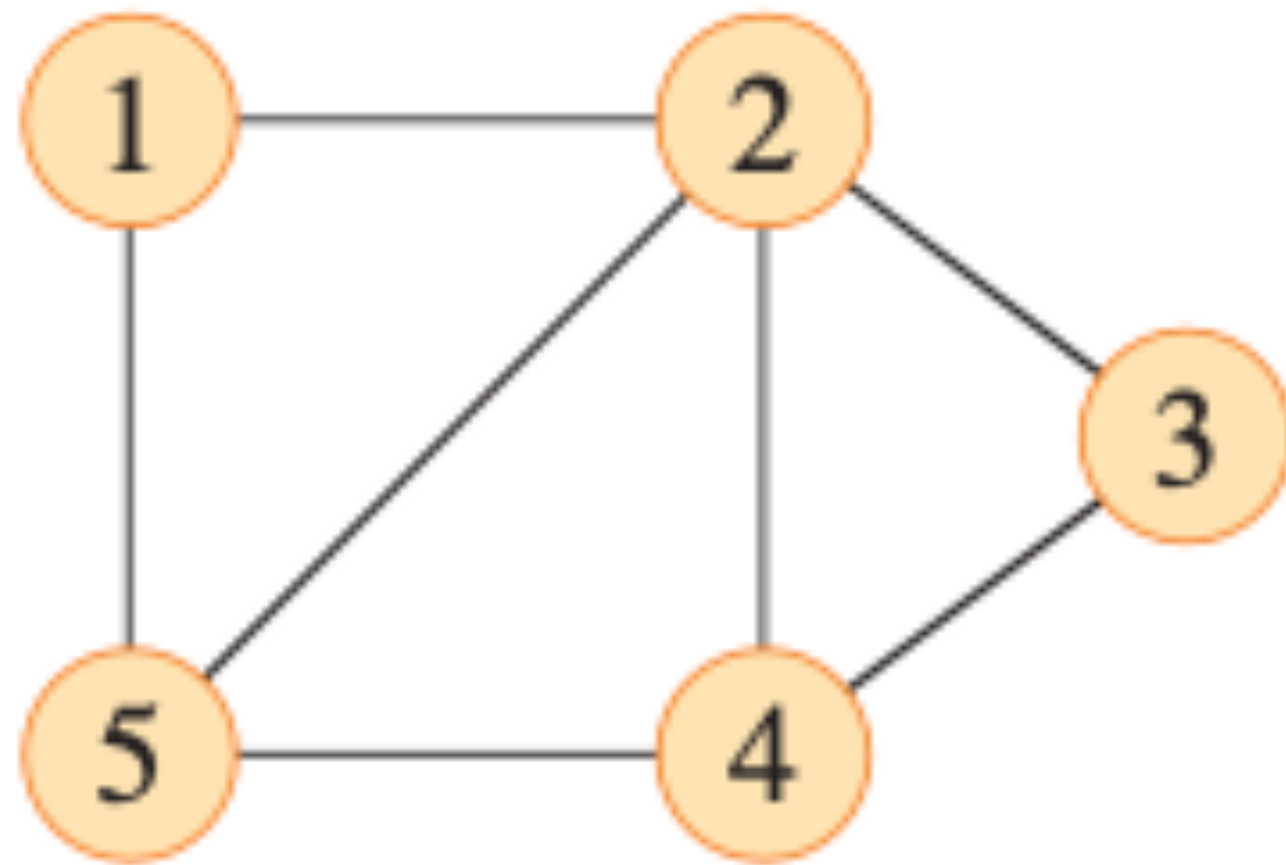
Row: an equation

Column: coefficients of a variable

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Graphs and networks



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Adjacency matrix

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Geometric points sets: Geometry and vectors

$n \times m$ matrix $\Rightarrow$ Cloud of n points (rows) in m dimensions (columns)

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Geometric points sets: Geometry and vectors

$n \times m$ matrix $\Rightarrow$ Cloud of n points (rows) in m dimensions (columns)

Dot product of two vectors:

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Geometric points sets: Geometry and vectors

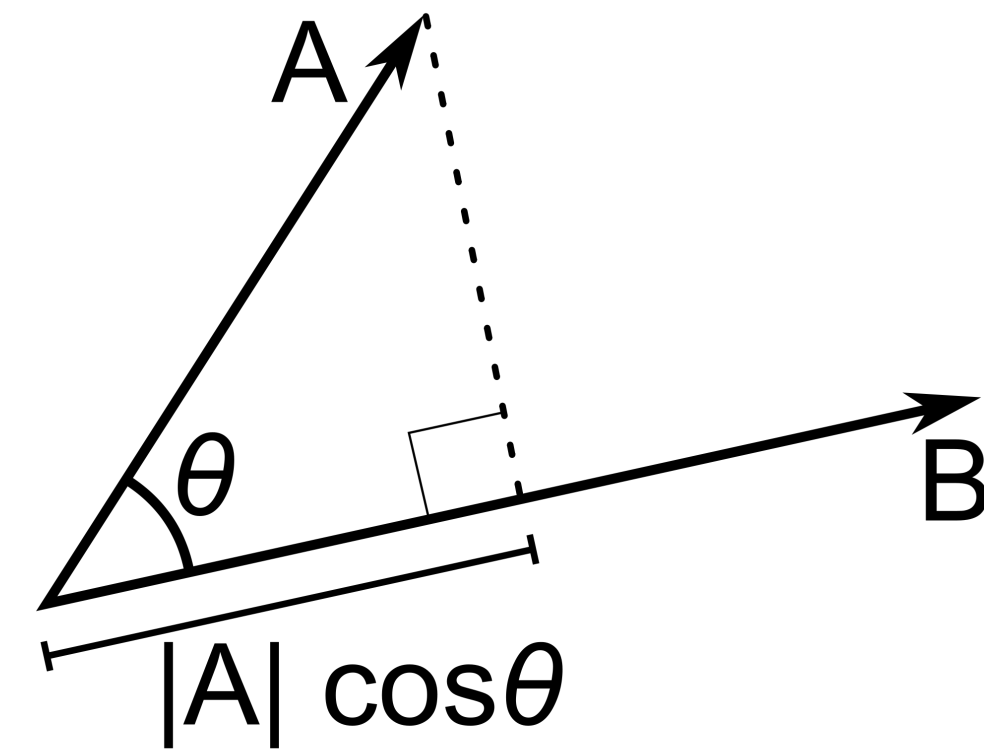
$n \times m$ matrix $\Rightarrow$ Cloud of n points (rows) in m dimensions (columns)

Dot product of two vectors:

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

Can compute the angle:

$$\cos(\Theta) = \frac{A \cdot B}{||A|| ||B||}$$



where $||A|| = \sqrt{\sum_{i=1}^n A_i^2}$ is the magnitude (length).

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Geometric points sets: Geometry and vectors

$n \times m$ matrix $\Rightarrow$ Cloud of n points (rows) in m dimensions (columns)

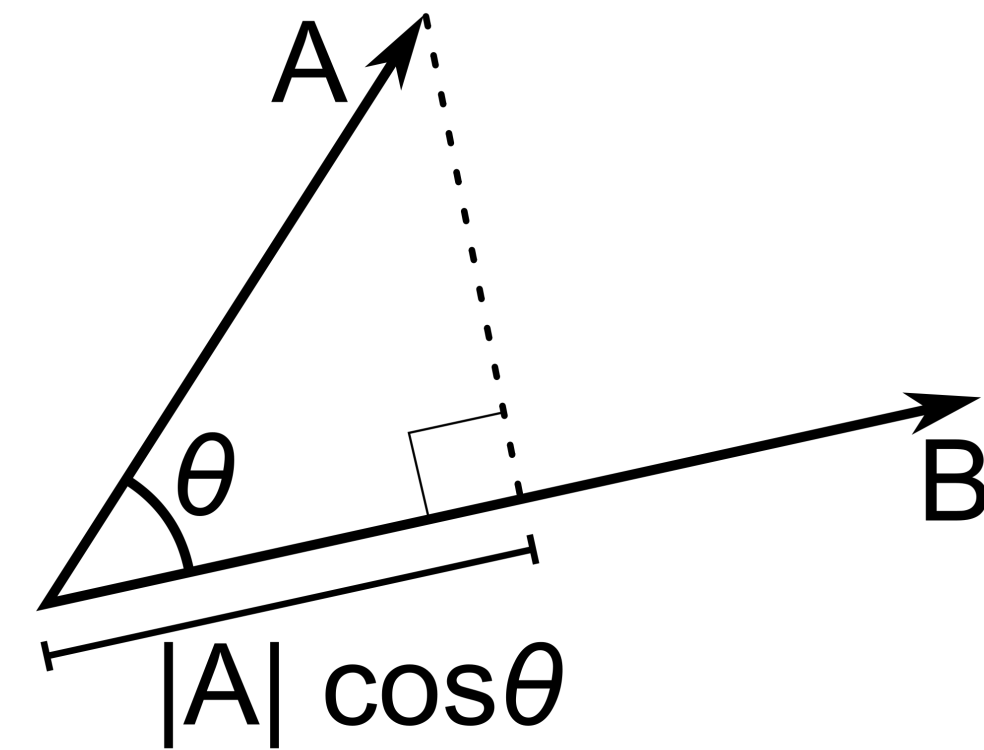
Dot product of two vectors:

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

Familiar?

Can compute the angle:

$$\cos(\Theta) = \frac{A \cdot B}{||A|| ||B||}$$



where $||A|| = \sqrt{\sum_{i=1}^n A_i^2}$ is the magnitude (length).

LINEAR ALGEBRA REVIEW

Matrix representations of important objects

- Geometric points sets: Geometry and vectors

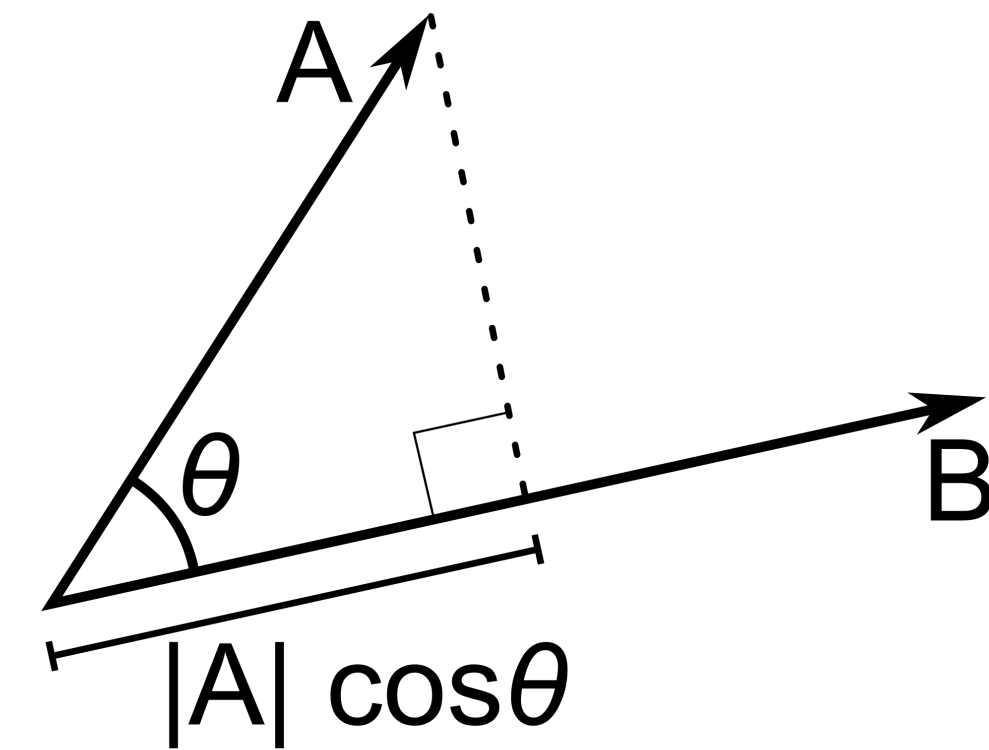
$n \times m$ matrix $\Rightarrow$ Cloud of n points (rows) in m dimensions (columns)

Dot product of two vectors:

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

Can compute the angle:

$$\cos(\Theta) = \frac{A \cdot B}{||A|| ||B||}$$



Familiar?

Cosine similarity: in $[-1, 1]$

(Similar to Pearson correlation coefficient - exactly the same when means are 0.)

where $||A|| = \sqrt{\sum_{i=1}^n A_i^2}$ is the magnitude (length).

MATRIX OPERATIONS

Matrix addition: $C = A + B$

$$C_{ij} = A_{ij} + B_{ij}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

Scalar multiplication: $A' = rA$

$$A'_{ij} = rA_{ij}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

MATRIX OPERATIONS

Matrix addition: $C = A + B$

$$C_{ij} = A_{ij} + B_{ij}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

Scalar multiplication: $A' = rA$

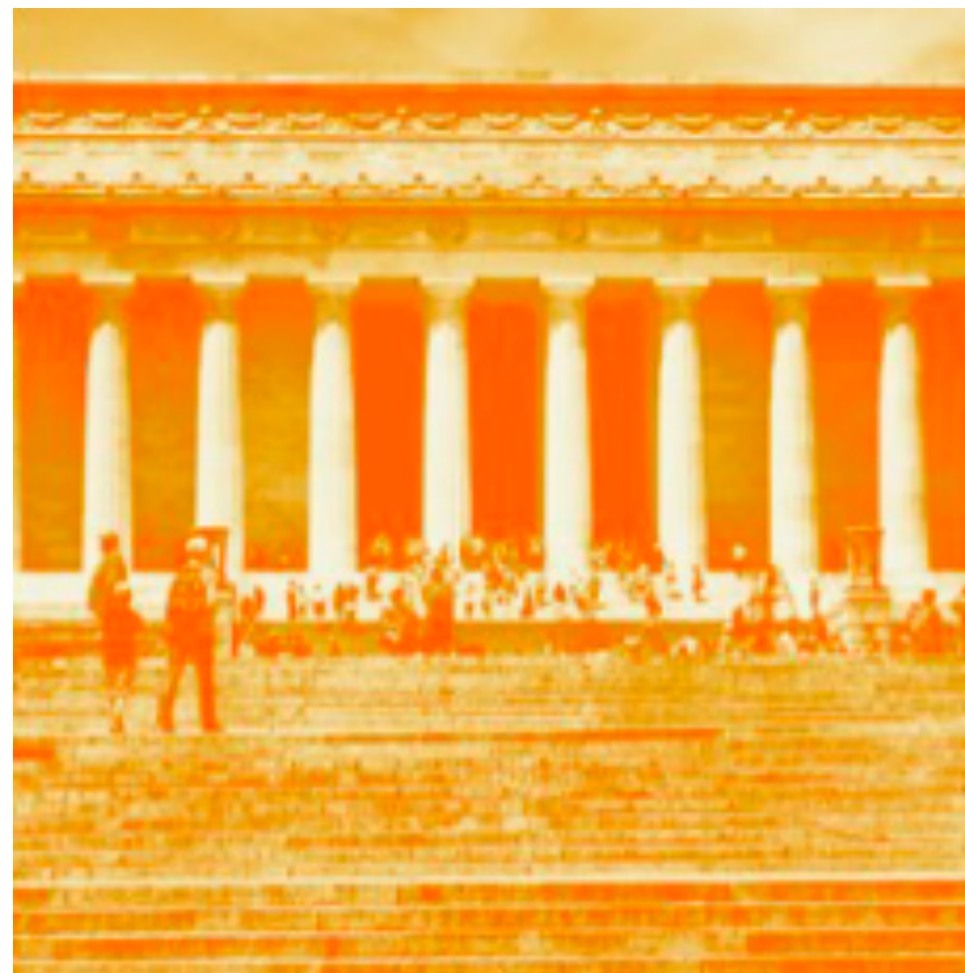
$$A'_{ij} = rA_{ij}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

Linear combinations: $\alpha A + (1 - \alpha)B$



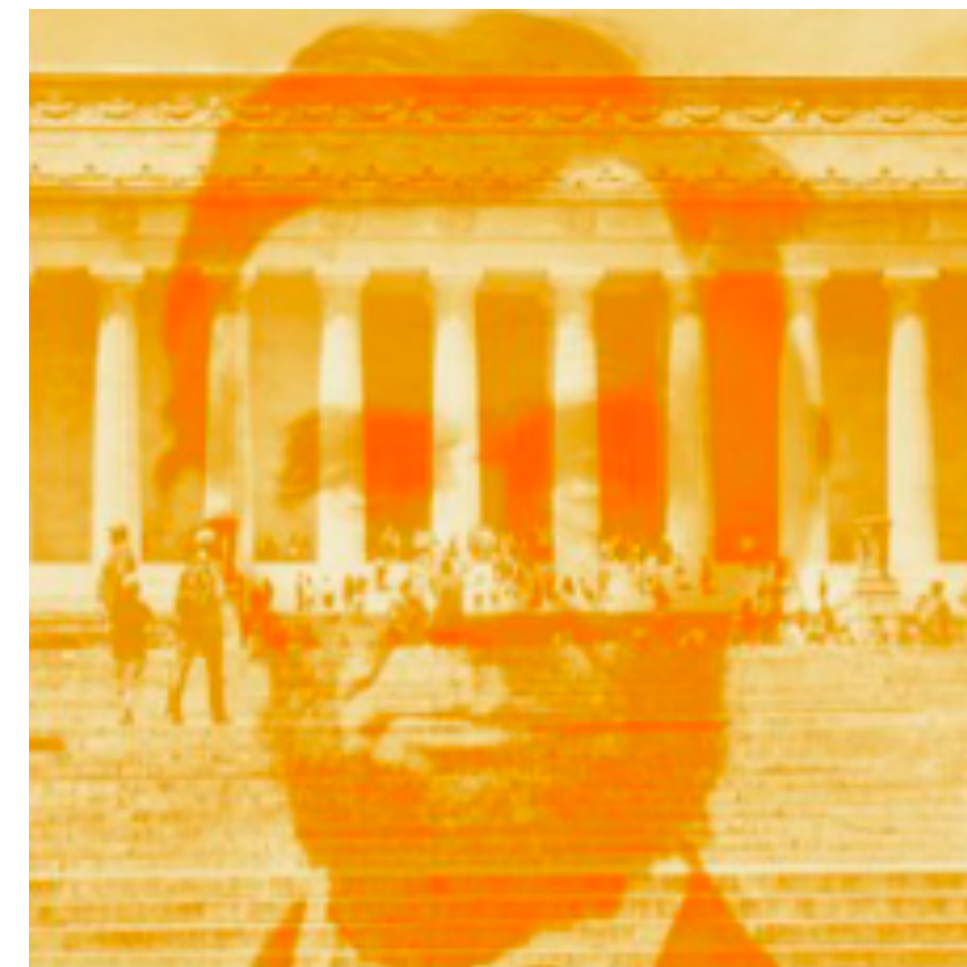
αL

+



$(1 - \alpha)M$

=



Lincoln&Memorial

for $\alpha = 0.5$

MATRIX OPERATIONS

Matrix transpose: A^T

$$A_{ij}^T = A_{ji} , \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

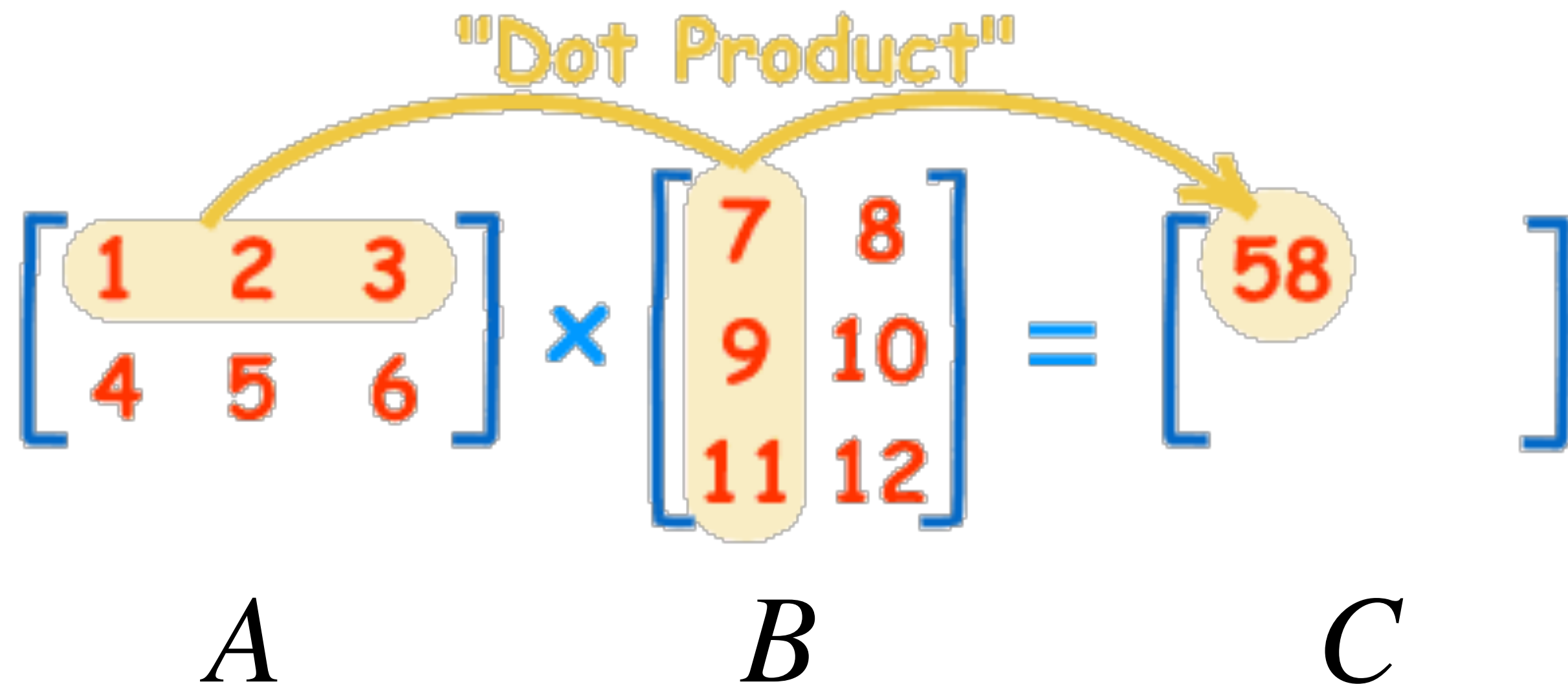
MATRIX OPERATIONS

Matrix transpose: A^T

$$A_{ij}^T = A_{ji}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

Matrix multiplication: $C = AB$ (A is $n \times k$, B is $k \times m$)

$$C_{ij} = A_i \cdot B_j, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$



Note that, Dot products measure how “in sync” the two vectors are (like covariance or correlation).

MULTIPLYING A MATRIX WITH ITS TRANSPOSE

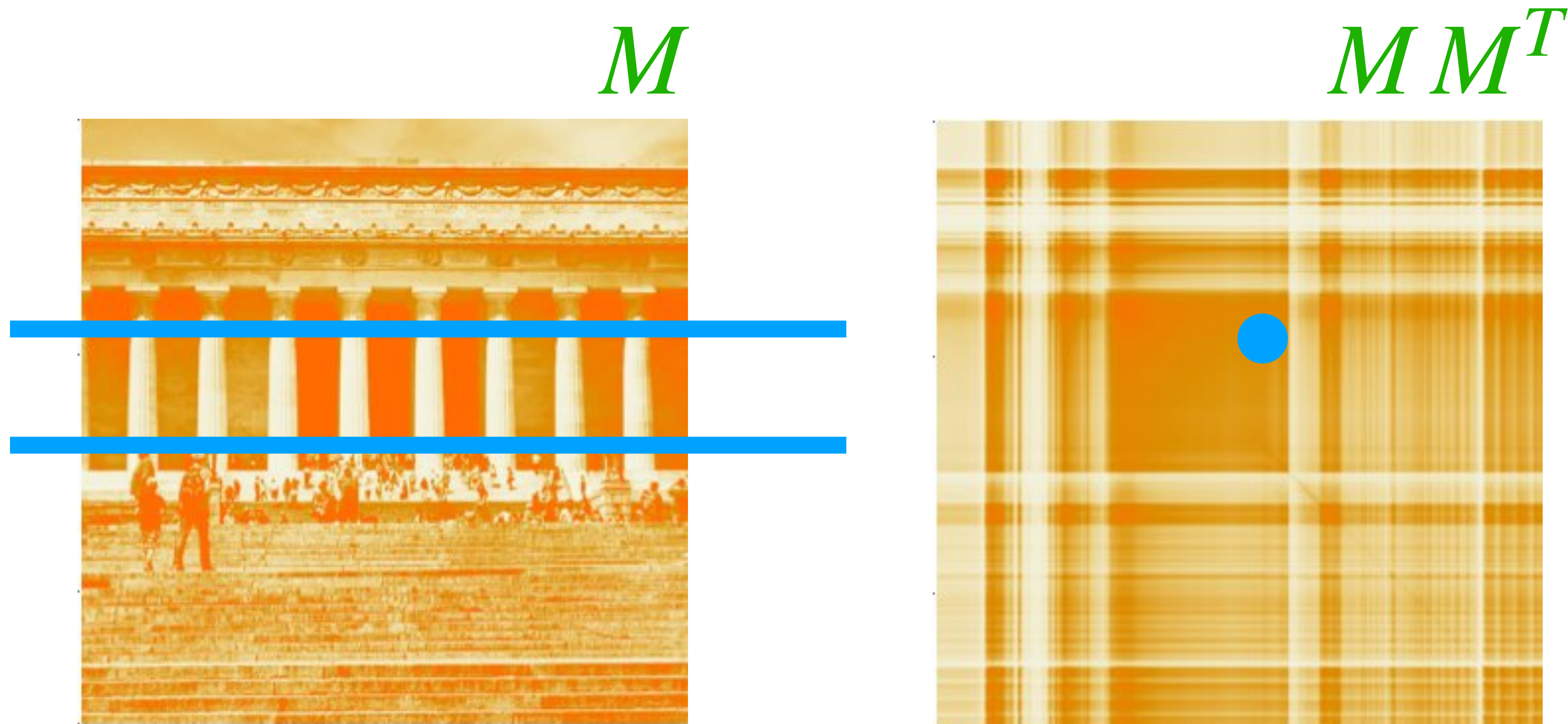
$A A^T$ measures “in-syncness” among rows (items/points).

$A^T A$ measures “in-syncness” among columns (features/dimensions).

MULTIPLYING A MATRIX WITH ITS TRANSPOSE

$A A^T$ measures “in-syncness” among rows (items/points).

$A^T A$ measures “in-syncness” among columns (features/dimensions).



MULTIPLYING A MATRIX WITH ITS TRANSPOSE

$A A^T$ measures “in-syncness” among rows (items/points).

$A^T A$ measures “in-syncness” among columns (features/dimensions).

